

TODO:

– Entscheidbarkeit

– Minimalautomat

– DEA/NEA/TM/NTM

– Beispiele:

– Kann eine NTM in polynomieller Zeit entscheiden, ob ein Ausfällrätsel eine Lösung hat? → Satz von Rice / P Verifizierer / Reduktion

– Ist Grammatik eindeutig? (FS_2026)5)

– Parse-tree finden (FS_2022)5))

– Prüfungsaufgaben:

– HS_2021)4)

– FS_2022)4)

Begriff	Definition
Regulär	Es gibt einen DEA A, der L akzeptiert, also $L(A) = L$
Kontextfrei	L kann nur von einem ND PDA erkannt werden

VORGEHEN ZU AUFGABENTYPEN

Ist Sprache regulär?

Ja: DEA / NEA / RegEx

Nein: Pumping Lemma

Ist Sprache kontextfrei?

Ja: Stackautomat / CNF / BNF

Nein: Pumping Lemma (CFL)

Turing-Maschinen

Ist eine Sprache Turing-Vollständig?

Kann Problem von einem Computer gelöst werden?

Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?

Kann eine nichtdeterministische Maschine ... entscheiden?

Warum kann ein Problem nicht gelöst werden?

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

– Zustände: $Q = \{q_0, q_1, \dots, q_n\}$

– Alphabet: Σ

– Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow Q$

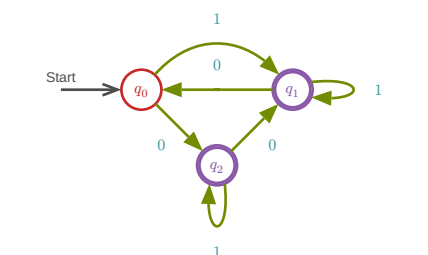
– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subset Q$

Q

F

0	1
q ₀	q ₂
q ₁	q ₀
q ₂	q ₁



PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache nicht regulär ist

- Vorgehen:
- Annahme: L ist regulär
 - Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
 - Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition muss N verwenden!

- Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 1 : $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = 0^{N+1} 1^{N+1}$
- Unterteilung $w = xyz$
- Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 0 bleibt
- $xy^kz \notin L$ für $k \neq 1$, Widerspruch zum Pumping Lemma

Beispiel 2 : L = alle korrekte Integralausdrücke

- Annahme: L ist regulär
- $\exists N \in \mathbb{N}$, Pumping Length
- $w = \int_{a_1}^{b_1} \int_{a_2}^{b_2} \dots \int_{a_N}^{b_N} dx_1 \dots dx_N$ in L
- Unterteile $w = xyz$, wobei $|y| \geq 1 \wedge |xy| \leq N \wedge xy^kz \in L$
- Pumpen: Der Teil xy enthält keine dx_k , beim pumpen nimmt also nur die Anzahl der Integralzeichen zu, so dass kein korrekter Integralausdruck stehen bleibt.
- Der Widerspruch zeigt, dass die Annahme, L sei regulär, nicht haltbar ist. Also ist L nicht regulär.

- 6 Schritte des Pumping-Lemma-Beweises: Annahme (A) 1 Punkt, Pumping Length (N) 1 Punkt, Wort (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Folgerung (F) 1 Punkt.

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

$A = (Q, \Sigma, \delta, q, F)$

– Endliche Menge von Zuständen: Q

– Alphabet: Σ

– Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$

– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subset Q$

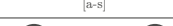
TODO: Thompson-NEA

VERALLGEMEINERTER NEA (VNEA)

Begriff	Definition
Regulärer Ausdruck	Zeichenkette r zur Beschreibung einer regulären Sprache $L = L(r)$
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 \mid r_2)$ $L(r_1) L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	NEA _s , dessen Übergänge mit regulären Ausdrücken beschriftet sind

PRIMITIVE REGULÄRE AUSDRÜCKE

Reguläre Ausdrücke für Wörter mit Länge ≤ 1

$L = L(r)$	r	NEA
$\emptyset$	$\emptyset$	Start 
$\{\varepsilon\}$	$\{\varearepsilon\}$	Start 
$\{a\}$	a	Start 
$\{o, s, t\}$	$[ost]$	Start 
$\{a, b, \dots, s\}$	$[a-s]$	Start 
Σ	$.$	Start 

KONTEXTFREIE SPRACHEN (CFL)

- $G = (V, \Sigma, R, S)$
- V: Variablen

– Σ : Terminalsymbole (Alphabet)

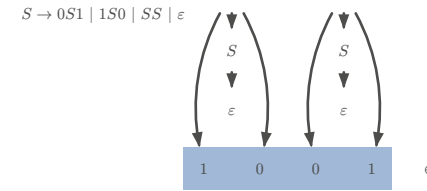
– R: Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$

– S: Startvariable

Parse Tree

$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$

mit der Grammatik



PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache nicht kontextfrei ist

Voraussetzungen:

- $|vy| > 0$
- $|vxy| \leq N$
- $uv^kxy^kz \in L \forall k \in \mathbb{N}$

Beispiel 3 : $L = \{w \in \{0, 1\}^* \mid w = 0^k 10^l 10^k 10^l\}$

- Annahme: L ist kontextfrei
- Nach dem Pumping Lemma $\exists N \in \mathbb{N}$, Pumping Length
- Wähle das Wort $w = 0^N 10^N 10^N 10^N$
- Nach dem Pumping Lemma gibt es eine Aufteilung von w in fünf Teile $w = uvxyz$ derart, dass $|vxy| \leq N \wedge |vy| \geq 1$. Ausserdem ist jedes gepumpte Wort $uv^kxy^kz \in L$.
- Da sich die Anzahl der Einsen beim Pumpen nicht ändern darf, müssen v und y vollständig in einem Nullen-Block enthalten sein. Daher kann sich nur die Anzahl der Nullen in höchstens zwei Nullen-Blöcken ändern. Die beiden Blöcke müssen wegen $|vxy| \leq N$ ausserdem benachbart sein.

Zum ersten Nullen-Block gehört der dritte, der gleich viele Nullen enthalten muss, zum zweiten gehört der vierte. Wie auch immer die beiden Blöcke gewählt werden, ändert sich die Anzahl Nullen in den Blöcken, aber nicht in den zugehörigen Blöcken. Das gepumpte Wort kann also nicht mehr in L sein.

6) Dieser Widerspruch zeigt, dass die Annahme, L sei kontextfrei, nicht haltbar ist. Also ist L nicht kontextfrei.

Pumping Lemma und Annahme L kontextfrei (PL) 1 Punkt, Pumping Length (N) 1 Punkt, Wahl eines Wortes (W) 1 Punkt, Unterteilung (U) 1 Punkt, Widerspruch beim Pumpen (P) 1 Punkt, Schlussfolgerung (S) 1 Punkt.

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$
- $S \rightarrow \epsilon$ ist erlaubt

UMWANDLUNG

- Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
- ϵ -Regeln:

$$\left. \begin{matrix} A \rightarrow \epsilon \\ B \rightarrow AC \end{matrix} \right\} \Rightarrow \left\{ \begin{matrix} B \rightarrow AC \\ \rightarrow AC \end{matrix} \right.$$

- Unit-Rules:

$$\left. \begin{matrix} A \rightarrow B \\ B \rightarrow CD \end{matrix} \right\} \Rightarrow \left\{ \begin{matrix} A \rightarrow CD \\ B \rightarrow CD \end{matrix} \right.$$

- Verkettungen:

$$A \rightarrow u_1 u_2 \dots u_n$$
$$A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$$

falls u_i ein Terminalsymbol: $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$

Beispiel 4 : $S \rightarrow ASA \mid aB, A \rightarrow B \mid S, B \rightarrow b \mid \epsilon$

1) Startvariable

$$\begin{matrix} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \epsilon \end{matrix}$$

2) ϵ -Regel

$$\begin{matrix} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \\ A \rightarrow B \mid \epsilon \mid S \\ B \rightarrow b \end{matrix}$$

3) Unit-Regel

$$\begin{matrix} S_0 \rightarrow S \\ S \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a \\ B \rightarrow b \end{matrix}$$

4) Komplexe Regeln

$$\begin{matrix} S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ S \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a \\ A_1 \rightarrow SA \\ U \rightarrow a \\ B \rightarrow b \end{matrix}$$

ABLEITUNGSDREIECK

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Startwort abgeleitet werden kann.

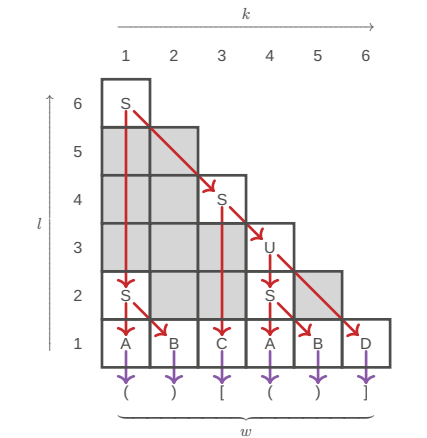
Beispiel

Parzen des Wortes $()([()])$

$$\begin{matrix} S \rightarrow AB \mid CD \mid CU \mid SS & B \rightarrow) \\ U \rightarrow SD & C \rightarrow [\\ A \rightarrow (& D \rightarrow] \end{matrix}$$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .



BNF

- Variablen: <variablen-name>
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: <variablen-name> ::= Ausdruck
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

STACKAUTOMAT (PDA)

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q: Zustände
- Σ : Eingabe-Alphabet
- Γ : Stack-Alphabet
- $\delta: Q \times \Sigma_e \times \Gamma_e \rightarrow P(Q \times \Gamma_e)$
- $q_0 \in Q$: Startzustand
- $F \subset Q$: Akzeptierzustände



a vom Input

b vom Stack entfernen (Bedingung)

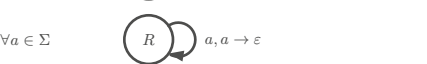
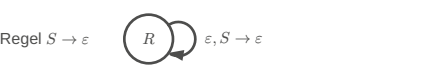
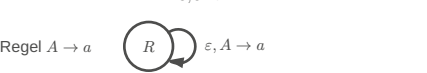
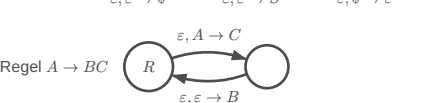
c auf den Stack

STACKAUTOMAT STANDARDISIEREN

TODO:

GRAMMATIK ABLESEN

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P, der L akzeptiert, $L = L(P)$.



$\forall a \in \Sigma$

$a, a \rightarrow \epsilon$

TODO:

2	Jedes Feld: Zeichen kommt in der Spalte nicht mehr vor	$O(n^4 \cdot n^2)$
3	Jedes Feld: Zeichen kommt im Unterquadrat nicht vor	$O(n^4 \cdot n^2)$
	Total	$O(n^6)$

NURIKABE

Entscheidbar? Ja: $n = uv$ = Anzahl Felder. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.

Zertifikat: Liste der schwarzen Felder

Vorgehen:

	Schritt	Aufwand
1	Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2	Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3	Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4	Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5	Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
	Total	$O(n^3)$

DAMEN

TODO:

REDUKTION

Polynomielle Reduktion: $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Zu beachten (Punkteverteilung):

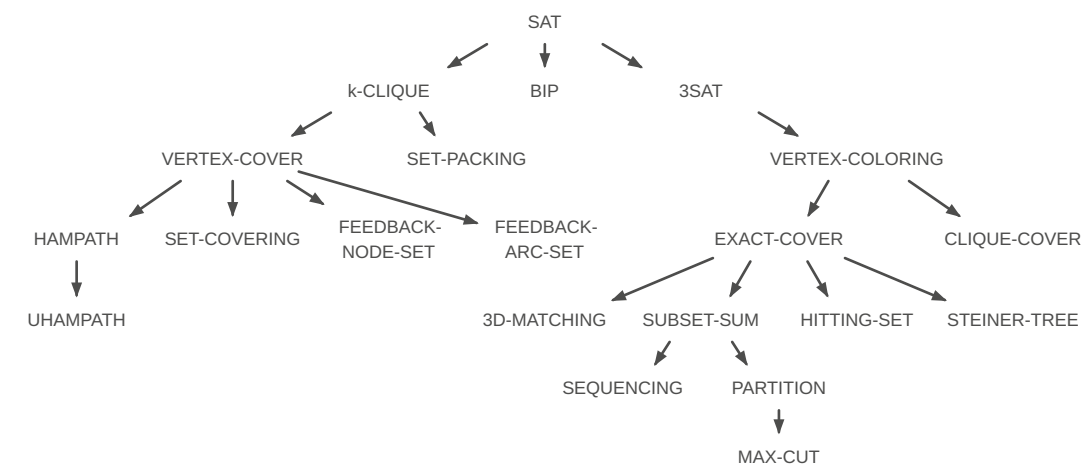
- Lösungsprinzip der Reduktion erwähnen (1P)
- Auswahl eines geeigneten Vergleichsproblems (1P)
- Reduktionsabbildung (*P)
- Schlussfolgerung: NP-Vollständig und somit nicht effizient lösbar (1P)

TODO:

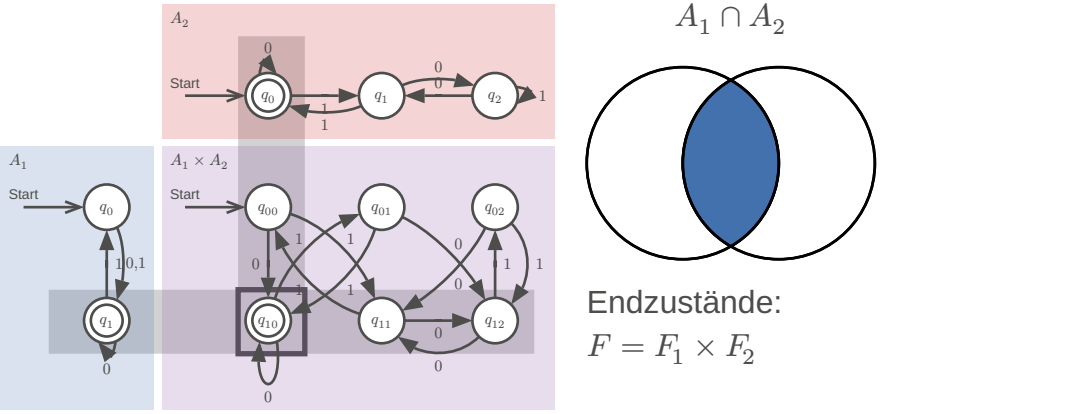
TURING-VOLLSTÄNDIG

Eine Sprache A heisst eine **Programmiersprache**, wenn es eine Abbildung $c : A \mapsto \Sigma^*$ gibt, wobei $c(w)$ ein Programm für eine universelle Turing-Maschine ist.

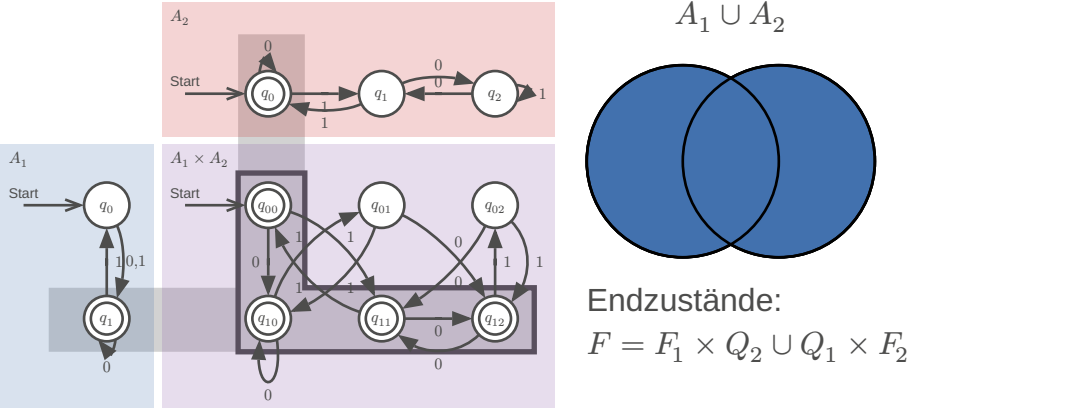
Eine Programmiersprache heisst **Turing-vollständig**, wenn in ihr jede beliebige Turing-Maschine simuliert werden kann.



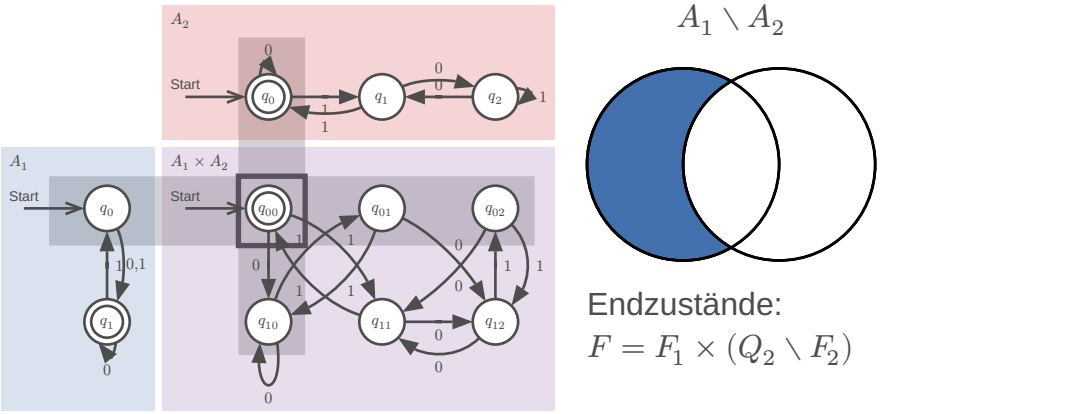
MENGEN
SCHNITTMENGE $L(A_1) \cap L(A_2)$



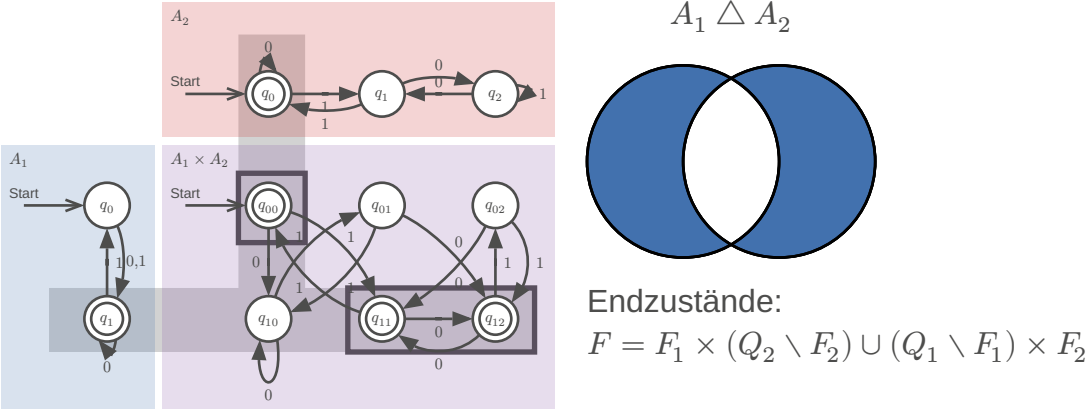
VEREINIGUNGSMENGE $L(A_1) \cup L(A_2)$



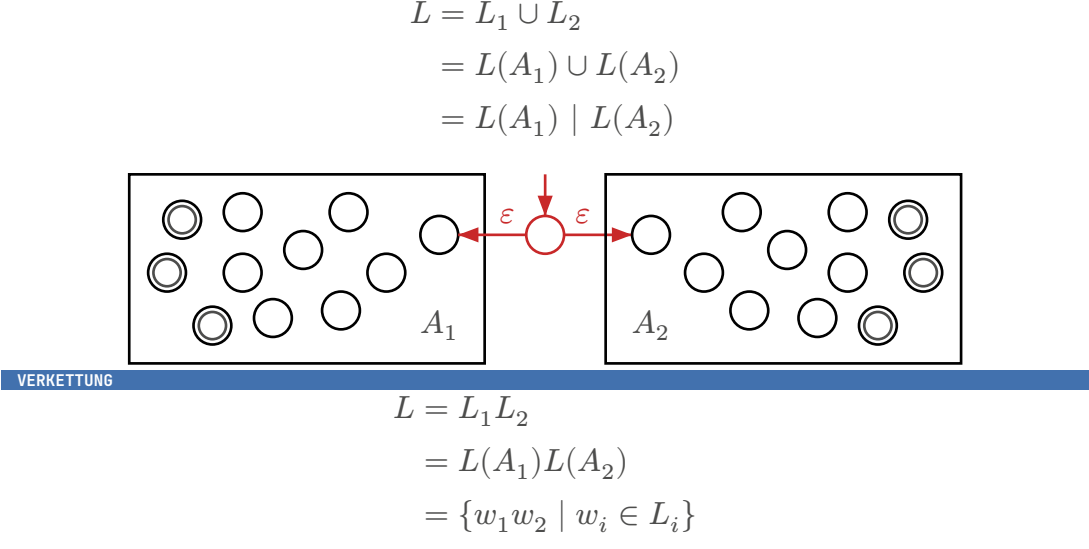
DIFFERENZMENGE $L(A_1) \setminus L(A_2)$

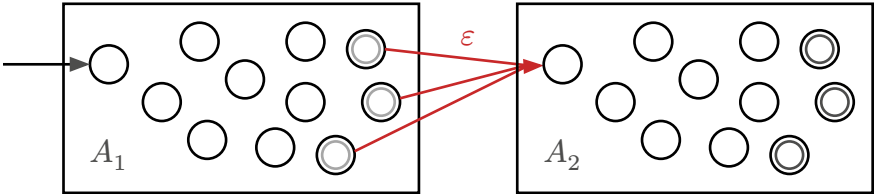


SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$



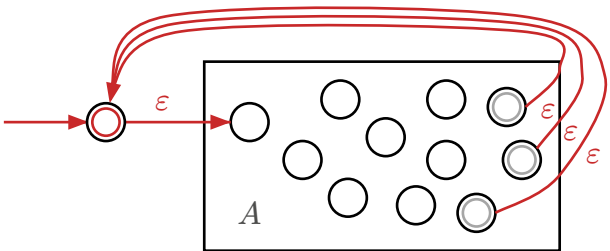
REGULÄRE AUSDRÜCKE
ALTERNATIVE





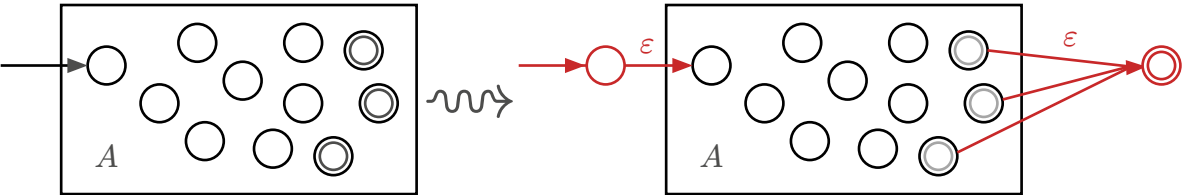
*-OPERATION

$$L^* = \{\epsilon\} \cup L \cup L^2 \cup \dots$$
$$= \bigcup_{k=0}^{\infty} L^k$$



VNEA ZU REGEX

Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Nach Entfernen aller Zwischenzustände $q_{rip} \in Q$ bleibt ein regulärer Ausdruck r von A

